OccupiedLength [SimTalk] - Track

Returns the part of the entire **Length** of the *Track* designated by <Path>, which is occupied by all *Transporters* located on it.

Remarks

Each *Transporter* located on the *Track* occupies part of the entire length available.

Type

Read-only attribute

Syntax

```
<Path>.OccupiedLength → length
```

Return Value

The return value has the data type *length*.

Example

```
print MyTrack.OccupiedLength
```

See also

Length [text box] - Track

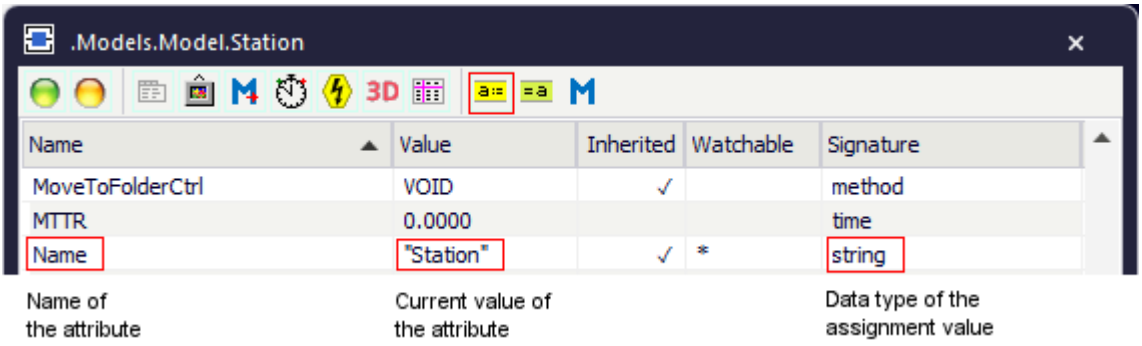
Attributes of the Track

_Attributes of the Track

The *Track* provides:

- The *attributes* listed in the table of contents to the left.
- The **Attributes of All Objects**.
- The **Attributes of the Material Flow Objects**.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class [general description]**.
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance [general description]**.

You can set the value of an attribute and you can get its value, either with the check boxes, the text boxes and drop-down lists in the dialog windows or by assigning values to the respective attributes.

- To **set the value** of an attribute, you might, for example, type:

```
MyTrack.Length := 44
```

- To **get the value** of an attribute, you might, for example, type:

```
print MyTrack.Length
posit := MyStation.Cont.XPos
```

BwDestList [SimTalk]

Sets the name of the destination list for a *Transporter* driving in reverse on the *Track* designated by <Path>.

Type

Attribute

Syntax

```
<Path>.BwDestList:object
```

Assignment Value

You can assign a value of data type *object*.

The destination list contains all destinations that the *Transporter* can reach while driving in reverse on this *Track*.

Example

```
MyTrack.BwDestList := myDataList
```

See also

Backward Destination List [text box]

Capacity [SimTalk] - Track

Sets the maximum number of *Transporters* that may be located on the *Track* designated by <Path> as a whole or in part at any one time.

Type

Attribute

Syntax

```
<Path>.Capacity:integer
```

Watchable

The attribute is **watchable**.

Assignment Value

You can assign a value of data type *integer*.

Specify `-1` for an infinite capacity.

Example

```
if MyTrack.Capacity = -1
   @.move(AEConveyor)
end
```

See also

Capacity [text box] - Track

FwDestList [SimTalk]

Sets the name of the destination list for *Transporters* driving forward on the *Track* designated by <Path>.

Remarks

The destination list contains all destinations that the *Transporter* can reach, while moving in the forward direction on this *Track*.

Type

Attribute

Syntax

```
<Path>.FwDestList:object
```

Assignment Value

You can assign a value of data type *object*.

Example

```
MyTrack.FwDestList := myDataList1
```

See also

Forward Destination List [text box]

Length [SimTalk] - Track

Sets the **Length** of the *Track* designated by <Path>.

Remarks

A *Transporter* enters the *Track* at position 0, and exits it after covering the length which you enter here. *Speed* and *Length* result in the dwell time on the *Track*.

Type

Attribute

Syntax

```
<Path>.Length:length
```

Watchable

The attribute is **watchable**.

Assignment Value

You can assign a value of data type *length*.

Note
In *SimTalk* 2.0 you can specify the length units `m`, `mm`, `km`, `cm`, `yd`, `ft`, and `in`.

Type in the unit directly after the values, without a separating blank space, for example `10m` or `10.2m`.

You can specify the unit for *floating point values* and for *integer values*.

Example

```
MyTrack.Length := 44m
```

SimTalk

OccupiedLength [SimTalk] - Track

See also

Length [text box] - Track

Width [SimTalk] - Track

Sets the **Width** of the *Track* designated by <Path>.

Type

Attribute

Syntax

```
<Path>.Width:length
```

Watchable

The attribute is **watchable**.

Assignment Value

You can assign a value of data type *length*.

> **Note**
> In *SimTalk* 2.0 you can specify the length units `m`, `mm`, `km`, `cm`, `yd`, `ft`, and `in`.
>
> Type in the unit directly after the values, without a separating blank space, for example `10m` or `10.2m`.
>
> You can specify the unit for *floating point values* and for *integer values*.

Example

```
MyTrack.Width := 2 // meters
```

See also

Width [text box] - AngularConverter

TwoLaneTrack

TwoLaneTrack

Use the object *TwoLaneTrack* for modeling a transport line with two lanes on which, with or without **automatic routing**, on which the *Transporter* moves parts in opposite directions.

Image

Description

You might, for example, use the *TwoLaneTrack* and the *Transporter*to model an AGV (automated guided vehicle) system. The distance, which the **Transporter** has to travel on the *TwoLaneTrack* is defined by the **Length** of **Lane A** and the **Length** of **Lane B** of the *TwoLaneTrack*, the *Transporter's* **MU Length**, and its **Speed**. They determine the time, which the *Transporter* remains on the *TwoLaneTrack*. As opposed to the point-oriented material flow objects, *Plant Simulation* uses the actual length, which you type in, during your simulation run. A *Transporter* may not pass another one moving in front of it. The *Transporters* thus retain their order of moving onto and of leaving the *TwoLaneTrack*. Each lane of the *TwoLaneTrack* may have its own length to realistically model the length of the lanes, when the *TwoLaneTrack* turns the corner. In that case the outside lane is longer than the inside lane.

If several *Transporters* travel along a lane of the *TwoLaneTrack* at a different speed, the faster one collides with the slower one. *Plant Simulation* then activates the **Collision Control** of the faster *Transporter* and automatically reduces its speed to the speed of the slower *Transporter*.

The *Transporter* can drive forward and **Backwards** on the *TwoLaneTrack*, i.e., it drives onto the *TwoLaneTrack* at its **exit** and exits it at its **entrance**. Driving forward and backward are not properties of the *TwoLaneTrack*, but of the *Transporters*, which drive on it.

The maximum capacity of the *TwoLaneTrack* is defined by its length and the lengths of the individual *Transporters* moving on it, i.e., a *TwoLaneTrack* that is three yards long accepts three *Transporters* of one yard each at the most. With the **Capacity** you can further restrict the number of *Transporters* located on the *TwoLaneTrack*.

You can insert the *TwoLaneTrack*:

- As a curved object, which is the default setting.
- By inserting any sequence of curved segments and straight segments, you can realistically model curved conveyor systems on which the *Transporters* move.

  You can select different configurations for the *TwoLaneTrack* on the tab **Appearance**.